AOSP 进阶丛书 · 第八册 · ANDROID 13 (API 33)

# 图形渲染深潜流水线全解

BufferSlot 四态状态机、BLAST 每帧原子事务、SurfaceFlinger 的 commit/composite 内景、HWC 直通决策、Fence 依赖图与帧率投票——一个像素从 App 到面板的完整白盒。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

# 目 录


前言：从"会用图"到"管流水线" . . . . . 2

第一章 全链路总览：一个像素的旅程 . . . . . 2

第二章 图形内存：Gralloc 与 GraphicBuffer . . . . . 3

第三章 BufferQueue：缓冲状态机 . . . . . 4

第四章 BLAST：把帧升级为原子事务 . . . . . 6

第五章 Surface 与 Transaction：App 侧的合同 . . . . . 7

第六章 SurfaceFlinger 内景：commit 与 composite . . . . . 8

第七章 合成决策：Overlay 直通 vs GPU . . . . . 10

第八章 Fence：跨设备时序的艺术 . . . . . 12

第九章 设计决策九问 . . . . . 12

附录 A 观测实操 . . . . . 13

附录 B 快问快答（18 条） . . . . . 13

附录 C 自检题 . . . . . 14

附录 D 动手实验室（一天量） . . . . . 14


**本册速览**

BufferSlot 四态状态机：dequeue/queue/acquire/release 逐行注释
BLAST：每帧一个原子事务（影子队列/三段回执/desiredPresentTime）
SF 内景：commit（latch 旧帧哲学）与 composite 两阶段
HWC 直通决策（validate→DEVICE/CLIENT）· Fence 依赖图 · 帧率投票（LayerHistory）
附录 D 五个实验室：状态机现场/直通侦查/fence 时轴/BLAST 背压/帧率投票

# 前言：从"会用图"到"管流水线"

第五册把画面交到了 SurfaceFlinger 门口就停了——本册推门进去：**缓冲如何轮转（BufferQueue 状态机）、帧如何原子提交（BLAST 事务）、SF 如何合成一帧（commit/composite）、硬件如何直通（HWC 决策）、跨设备的时序如何对齐（Fence）**。这五件事是图形岗面试与调优的全部主战场。

体例与前册一致：真实源码逐行注释 + 完整调用链 + 流程图。源码基线不变（本树 Android 13）；GUI=frameworks/native/libs/gui、SF=frameworks/native/services/surfaceflinger。读码路线：状态机（第三章）→ 事务（第四章）→ 合成（第六七章）→ fence（第八章），每章是上一章的下一段流水。

# 第一章 全链路总览：一个像素的旅程




图 1 图形渲染全链路：App 生产→BLAST 提交→SF 合成→HWC 送显四段，含双 VSync 与三种旅行方式

四段流水各有一位"所有者"：App 拥有生产节拍（Choreographer，书5 § 四），BLAST 拥有提交语义（事务），SF 拥有合成世界（Layer 树），HWC 拥有最终带宽（直通/叠加）。**四段之间不传像素指针不等待，只传两样东西：缓冲句柄与 fence**——这是整册的第一心法。

# 第二章 图形内存：Gralloc 与 GraphicBuffer

一切从一块内存开始：GraphicBuffer（GUI/GraphicBuffer.cpp）经 **Gralloc**（hardware/interfaces/graphics/{allocator,mapper}，AIDL 化）分配——allocator HAL 出 dma-buf 句柄，mapper HAL 负责跨进程映射（lockAsync）。三个读码要点：

- **usage bits 决定内存去向**（GPU 纹理/扫描输出/编码器输入各走各的堆）——视频层能 Overlay 的物理前提是 buffer 落在控制器可见的堆；
- **跨进程零拷贝的真相**：同一 dma-buf 被 App（画）、SF（合成）、控制器（扫描）各自映射——**一份物理内存三个视图**；
- stride（行距）与格式（RGBA8888/P010…）： mapper 层概念泄漏到用户的常见坑（读图像出现斜纹= stride 算错）。

**源码锚点 · 第二章锚点**

<font face="MonoFT" size="9">GUI/GraphicBuffer.cpp</font>：allocate/lockAsync 封装<br/><font face="MonoFT" size="9">hardware/interfaces/graphics/allocator/aidl</font>：分配 HAL<br/><font face="MonoFT" size="9">hardware/interfaces/graphics/mapper/aidl</font>：映射 HAL<br/><font face="MonoFT" size="9">hardware/interfaces/graphics/bufferqueue/2.0</font>：13 新 AIDL BQ 试点（媒体域直连）

## 2.1 GraphicBuffer 的一生（逐段注释）

cpp GUI/GraphicBuffer.cpp（骨架+讲解注释）

```
// 讲解注释①：构造仅记尺寸/格式/usage——懒分配：
// 真正的分配在首次 dequeue 需要"新槽"时（BufferQueueCore 请求）。
// GraphicBuffer::allocate():
// // 讲解注释②：走 Gralloc allocator（AIDL）：
// // "我要 width×height 的 P010、usage 含 GPU_SAMPLED+COMPOSER_OVERLAY"
// // HAL 返回 bufferId + handle（dma-buf 句柄）——内存在这里诞生。
// GraphicBuffer::lockAsync(..., outFence):
// // 讲解注释③：mapper HAL 按当前进程映射 + 返回可写地址（带 fence 依赖）；
// // 跨进程方（SF/控制器）各自 lockAsync 同一 bufferId——同一份物理页。
// GraphicBuffer::unlockAsync():
// // 讲解注释④：产生新的 acquire fence（GPU/驱动的写完承诺）。
```

**设计洞察 · 逐段精读：懒分配的用意**

分配是昂贵且可能失败的操作（ION/dmabuf 池有限）；把分配推迟到"确实需要第 N 个缓冲"，池就能随帧率需求自适应——慢速 UI 两块就够，视频三块，录屏再加。dumpsys SurfaceFlinger --dump-buffer 里看到的槽位增长史就是这段逻辑的现场。

# 第三章 BufferQueue：缓冲状态机

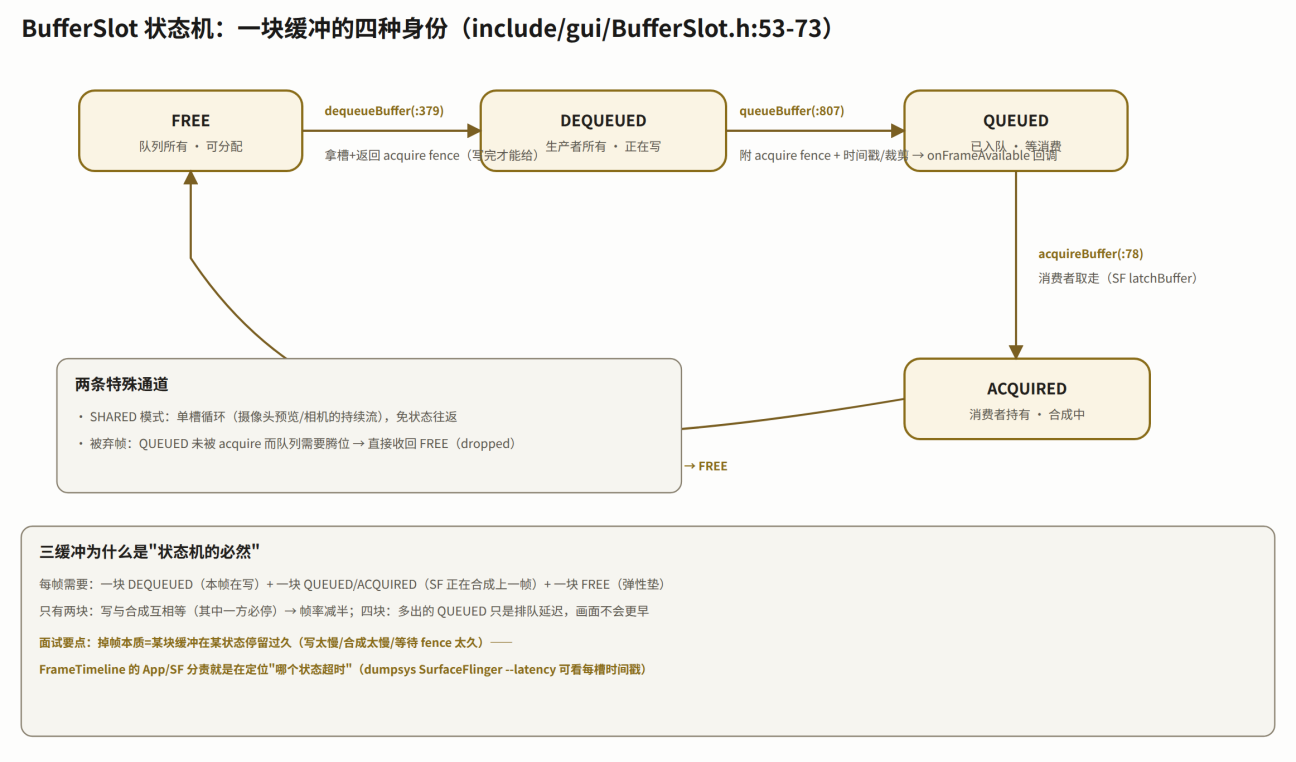



图 2 BufferSlot 状态机：FREE/DEQUEUED/QUEUED/ACQUIRED 四态流转与 SHARED 特殊通道，三缓冲的必然性

## 3.1 生产者两步逐行注释（真实源码）

cpp GUI/BufferQueueProducer.cpp:379-382 起（骨架+讲解注释）

```
status_t BufferQueueProducer::dequeueBuffer(int* outSlot,
sp<android::Fence>* outFence, uint32_t width, uint32_t height,
PixelFormat format, uint64_t usage, uint64_t* outBufferAge,
FrameEventHistoryDelta* outTimestamps) {
ATRACE_CALL();
// 讲解注释①：与消费者（SF）同锁——槽位表是两端共享账本，
// 这把锁就是"生产消费互斥"的全部实现（无内核对象）。
{ std::lock_guard<std::mutex> lock(mCore->mMutex);
...
// 讲解注释②：找 FREE 槽（或可复用的同尺寸槽）；没有则按
// maxDequeuedBufferCount 决定阻塞或返回 WOULD_BLOCK——
// App 侧 dequeue "卡住"的机制层就是这里在等 release。
...
// 讲解注释③：返回 slot + outFence（该槽上一任读者的 release
// fence）：GPU 编队时挂上依赖——fence 未到就写=花屏。
...
```

cpp GUI/BufferQueueProducer.cpp:807 起（骨架+讲解注释）

```
status_t BufferQueueProducer::queueBuffer(int slot, ...) {
// 讲解注释④：槽位 DEQUEUED→QUEUED；登记 acquire fence、
// crop/scale/时戳——这一帧的"自我描述"全部入账。
```

```
// 讲解注释⑤：唤醒消费者：mCore->mDequeueCondition /
// onFrameAvailable 回调（BLAST 模型里就是 BBQ:635 的入口）。
// 讲解注释⑥：返回 FrameEventHistoryDelta——帧时间线回执
//  （书5 FrameTimeline 的 App 侧数据源）。
}
```

## 3.3 BufferQueueCore：账本与三条队列

text GUI/bufferqueue/BufferQueueCore（结构+讲解注释）

```
mSlots[64] // 槽位数组上限 64——物理账本
mFreeSlots // 空槽（从未分配或已重置）——可触发新分配
mFreeBuffers // 已分配且 FREE 的缓冲——复用免分配（懒分配的另一半）
mQueue // QUEUED 帧队列（FIFO； acquire 侧按序取）
mDequeueCondition // 背压等待点（waitForFreeSlotThenRelock 的床）
mAbandoned // 消费者死了（SF 侧 Surface 销毁）→ 生产端 dequeue 报 NO_INIT
// 讲解注释：mFreeSlots 与 mFreeBuffers 的区分就是"要不要分配"的区分——
// 复用已分配缓冲零成本，所以稳态下 dequeue 永远走 mFreeBuffers。
```

**设计洞察 · 逐段精读：maxDequeuedBuffers 的谈判**

生产者可请求提高在途上限（IGraphicBufferProducer::setMaxDequeuedBufferCount），但需消费者点头（SF 按内存预算裁决）——三缓冲不是写死的三，是**上限谈判后的稳态**。SurfaceView 常配更多在途（视频解码超前），聊天列表两块即稳。

**设计洞察 · 逐段精读：一把互斥锁的两面**

①的共享账本意味着：dequeue 与 SF 的 acquire/release 在**同一进程空间**（BQ 的两端只在 App↔SF 之间——BINDER 只用于建立与控制，数据面不走 binder）。跨进程的生产消费（媒体编解码→App 显示）是 13 新 AIDL BQ 试点要解的题（第二章锚点）。记住分层：**同进程用锁，跨进程用事务，跨设备用 fence**。

## 3.1.1 dequeueBuffer 找槽段逐行注释（真实源码）

cpp GUI/BufferQueueProducer.cpp:411-446（节选+讲解注释）

```
{ // Autolock scope
std::unique_lock<std::mutex> lock(mCore->mMutex);
// 讲解注释⑤：无空闲槽且他人在分配中→等待（分配串行化，
// 防止并发分配同一池）——"dequeue 卡住"的第二种现场。
if (mCore->mFreeBuffers.empty() && mCore->mIsAllocating) {
mDequeueWaitingForAllocation = true;
mCore->waitWhileAllocatingLocked(lock);
...
}
// 讲解注释⑥：usage 合并消费者的要求（SF 侧声明"我要能直通的堆"）——
// 生产者未声明的能力由消费者补齐，分配才落在正确的物理堆。
usage |= mCore->mConsumerUsageBits;
// 讲解注释⑦：默认尺寸 + 旋转 90° 时宽高互换（transform hint 预转）。
...
int found = BufferItem::INVALID_BUFFER_SLOT;
while (found == BufferItem::INVALID_BUFFER_SLOT) {
// 讲解注释⑧：核心找槽循环：FREE 优先；不足则按
// maxDequeuedBufferCount 决定阻塞等 release 或丢弃最老 QUEUED。
```

```
status_t status = waitForFreeSlotThenRelock(FreeSlotCaller::Dequeue, lock, &found);
...
```

逐段精读⑤与⑧的两种等待语义：等**分配**（罕见、一次性）vs 等 **release**（常态、背压机制）。当 App 生产过快（游戏高帧率），⑧就是把生产速度压回消费速度的节流阀——**背压不在协议里，在状态机里**。

### 3.2 消费者两步

`BufferQueueConsumer::acquireBuffer`（:78）：按序取 QUEUED 帧（acquire fence 一并交出，SF latch 后在 composite 时真正等待）；`releaseBuffer`：挂 release fence 归还 FREE——**归还≠立即可写**，要等消费者读完（fence 才是最终话语权）。SHARED 模式（BufferSlot.h 注释末段）：单槽循环供相机持续流，免状态机往返。

## 第四章 BLAST：把帧升级为原子事务




图 3 BLAST 模型：旧 BQ 直连的三问题与每帧一事务的新世界，帧责任链三段回执

### 4.1 onFrameAvailable 逐段注释（真实源码骨架）

cpp GUI/BLASTBufferQueue.cpp:635 起（骨架+讲解注释）

```
void BLASTBufferQueue::onFrameAvailable(const BufferItem& item) {
// 讲解注释①：每帧独立 Transaction（循环取数复用，池化）；
// BBQ 自持一个"阴影 SurfaceControl"代表本窗口内容。
// 讲解注释②：t->setBuffer(...) 挂缓冲+acquire fence;
```

```
// setTransactionCompleteCallback → 帧三段回执
// （latched/presented/最终 released）——FrameTimeline 原料。
// 讲解注释③：t->setDesiredPresentTime：未来帧可预约
// （游戏帧 pacing 的官方路径）；
// 讲解注释④：t->apply() 提交（SurfaceComposerClient:958）
// → SF 事务队列；本函数返回≠帧生效，等回执。
}
```

update（:205）则处理窗口几何变化（尺寸/变换）——**几何与内容终于同车**。旧模型（≤11）内容走 BQ、几何走 WMS 两条道导致"新内容旧位置"闪烁；BLAST 把帧定义为原子单元，是 12 以来动画/多窗正确性的地基。

### 4.1.1 onFrameAvailable 真实逐段注释（源码 :635-690）

cpp GUI/BLASTBufferQueue.cpp:635 起（节选+讲解注释）

```
void BLASTBufferQueue::onFrameAvailable(const BufferItem& item) {
{
std::unique_lock _lock{mMutex};
const bool syncTransactionSet = mTransactionReadyCallback != nullptr;
// 讲解注释①：同步事务模式（WMS 与 BBQ 的同步握手）：
// 若上一同步事务里还占着 buffer，先 getAndClear+releaseBuffer
// 腾出槽——避免新帧无槽可用。
if (syncTransactionSet) {
if (!mAcquireSingleBuffer) {
auto bufferData = mSyncTransaction->getAndClearBuffer(mSurfaceControl);
if (bufferData) { releaseBuffer(...); }
}
}
// 讲解注释②：影子队列记账：mNumFrameAvailable++;
// 若在等回调且积压≥2，主动 acquireAndReleaseBuffer 丢弃旧帧
// ——BLAST 也有"丢帧保节拍"的快速通道（对应状态机的 dropped）。
mNumFrameAvailable++;
if (mWaitForTransactionCallback && mNumFrameAvailable >= 2) {
acquireAndReleaseBuffer();
}
...
```

**设计洞察 · 逐段精读：影子队列是什么**

BLAST 在 App 进程里自建一层小队列（shadow queue）：帧先入此队，再按节拍逐帧组 Transaction 提交 SF——**提交速率与生产速率解耦**。这解释了两个现象：动画卡时 App 仍流畅生产（影子队垫着）；连续两帧积压时旧帧被主动丢弃（宁可跳帧不排队延迟）。

# 第五章 Surface 与 Transaction：App 侧的合同

App 拿到的 Surface（GUI/Surface.cpp）是"带队列的画布代言"：lockCanvas（软件）/经 Surface 的 GPU 目标（硬件）最终都落到 dequeue→queue。**SurfaceControl 才是"窗口属性遥控器"**（书5 已见 WMS 侧），而 SurfaceComposerClient::Transaction::apply（:958）是两世界之间的签收台：buffer、几何、动画参数（书5 §8 的 RenderThread 动画就在这里生效）打包成一次原子

提交。

```
text apply 的三重语义（读码后总结）
原子性：多参数一次生效（无中间态可见）
异步性：入 SF 队列即返回，生效在下个 commit
可回执：TransactionComplete回调链（BLAST 三段）供帧归因
```

## 5.1 事务的批处理与合并

```
text Transaction 的三种用法（读码归纳）
单帧提交（BLAST）：buffer+几何+回执一车——每帧一次
纯属性提交（动画）：只改矩阵/alpha（RenderThread 动画）——可多帧高频
批合并：Transaction::merge / SF 侧事务队列按 vsyncId 分桶，
同帧内多个来源（WMS 几何+App 内容+动画）在 commit 时一次性结算
// 讲解注释：合并的单位是"同一 vsync 周期"——迟到者进下一桶，
// 这就是 apply 返回≠生效、且动画与内容永远同帧生效的机制保证。
```

## 6.3 多屏与 VirtualDisplay

每块屏一个 DisplayDevice + 独立 composite（注释十二：帧率策略 per-display）；**VirtualDisplay**（投屏/录屏/副屏模拟）是"没有面板的屏"：composite 输出到自己的 BufferQueue（由请求方消费），节拍自定（录屏常 30fps 桶）。排障提示：投屏卡顿别查主屏 vsync——查 VirtualDisplay 的消费者（投屏协议端）是否及时 release。**镜像**（secure display）则是主屏 composite 的旁路复制，成本最低但布局同主屏。

> **源码锚点 · 第五章锚点**
>
> <font face="MonoFT" size="9">GUI/SurfaceComposerClient.cpp:958</font> Transaction::apply<br/><font face="MonoFT" size="9">GUI/Surface.cpp</font>：生产者封装（dequeue/queue 的 App 门面）<br/><font face="MonoFT" size="9">GUI/BLASTBufferQueue.cpp:205/635</font>：update/onFrameAvailable

# 第六章 SurfaceFlinger 内景：commit 与 composite




图 4 SF 一帧内景：唤醒取件→commit（事务+latch+脏区）→composite（合成计划→HWC→present），Layer 两面与离屏层

`composite`（SF/SurfaceFlinger.cpp:2164）每 SF VSync 执行；`commitTransactionsLocked`（:3119）结算本帧世界：应用 Layer 几何、**latchBuffer**（取每层最新 QUEUED 缓冲，acquire fence 未到就用旧帧——帧率优先于新鲜度）、计算脏区。`commitOffscreenLayers`（:3512）单独结算不挂屏的层（录屏/虚拟屏走自己的节拍）。

## 6.1 VSync 分发逐段注释（Scheduler/EventThread.cpp:434 起）

cpp SF/Scheduler/EventThread.cpp:434-470（节选+讲解注释）

```
void EventThread::threadMain(std::unique_lock<std::mutex>& lock) {
while (mState != State::Quit) {
// 讲解注释①：事件来源三种：HOTPLUG（屏插拔，重置 vsync 态）、
// VSYNC（节拍本体）、MODE_CHANGE。逐个从 pending 队列出。
if (!mPendingEvents.empty()) {
event = mPendingEvents.front(); mPendingEvents.pop_front();
switch (event->header.type) {
case DISPLAY_EVENT_HOTPLUG: ... // 讲解注释②：热插拔时
case DISPLAY_EVENT_VSYNC: ... // 建立/重置 mVSyncState
}
}
// 讲解注释③：遍历连接（connection）按各自 vsyncRequest 派发：
// App 侧的 DisplayEventReceiver 即某条 connection 的 BitTube
// 对端（:179 stealReceiveChannel 交给 Choreographer 世界）。
// 请求分级：None/Single/Periodic——按需唤醒，没人要就不发。
...
```

逐段精读③的"按需分发"：EventThread 不是广播塔而是**点播台**——App 只有注册了 vsync 回调（Choreographer postCallback 之后）才会收到，SF 自己的调度也走同类机制（另一 EventThread 实例）。这是"没人监听的节拍不产生开销"的节能设计，也解释了 idle 时 perfetto 里 vsync 轨稀疏的原因。

## 6.2 composite 骨架逐段（:2164 起）

cpp SF/SurfaceFlinger.cpp:2164（骨架+讲解注释）

```
void SurfaceFlinger::composite(nsecs_t frameTime, int64_t vsyncId) {
// 讲解注释①：时间戳与 vsyncId 贯穿全帧——FrameTimeline 归因的
// 关联键（一次 composite 对应一组 presented 帧）。
// 讲解注释②：CompositionEngine::present(CompositionRefreshArgs)：
// 把"本帧 Layer 快照 + 输出屏集合"交给策略引擎（CE 层），
// SF 自身只管世界状态，不直接碰 HWC——职责分离的合成器版。
// 讲解注释③：结束后 sampleFrames? （LayerHistory 采样本帧各层
// 提交间隔）→ 帧率策略更新（自适应刷新的输入）。
// 讲解注释④：present fence 回收 → 通知各 Layer 的 releaseCallback
// → BLAST 三段回执的 presented 段在此触发。
}
```

**设计洞察 · 逐段精读：为什么 latch 不等 fence**

GPU 写完的 acquire fence 可能还要几十微秒；若 latch 阻塞等待，SF 主循环卡住，**整帧**延迟——代价是本层这帧用旧内容（stale frame，肉眼几乎无感）。这是"内容新鲜度 vs 帧节拍稳定"的经典取舍，也是 fence 设计成"事后声明"而非"事前同步"的原因（第八章呼应）。

# 第七章 合成决策：Overlay 直通 vs GPU

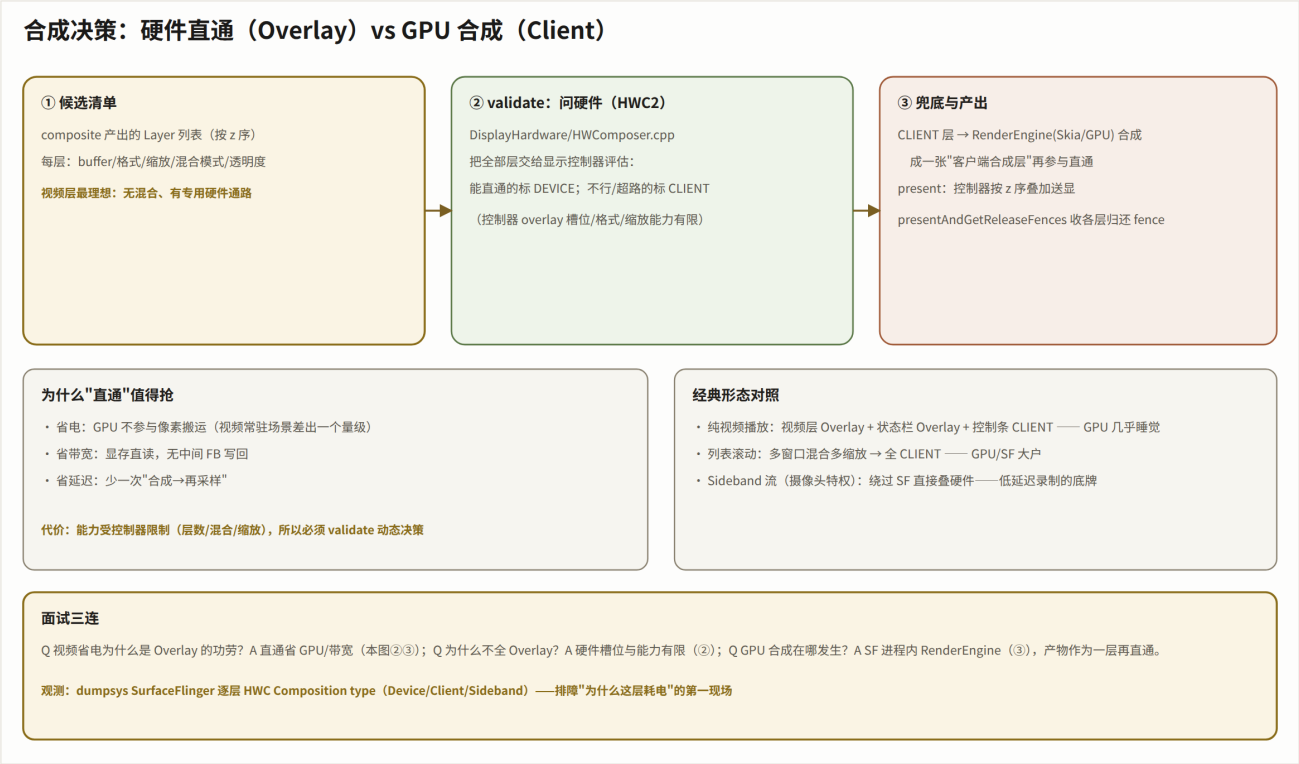



图 5 HWC 合成决策：候选→validate 硬件点名→CLIENT 兜底三步，省电三理由与经典形态对照

三步语义：composite 产出 Layer 清单 → `HWComposer`（SF/DisplayHardware/）validate 让控制器挑可直通层（标 DEVICE）→ 余层（CLIENT）经 **RenderEngine**（Skia GPU）合成一张再参与直通 → present 送显并收 present fence。决策变量：混合/缩放/格式/层数上限——**所以必须逐帧动态决策**而非静态分配。Sideband 流是特权旁路（相机低延迟录制）。

## 7.1 帧率策略：LayerHistory 投票（13 主线特性）

`Scheduler/LayerHistory.cpp` 持续采样每层"两次提交的间隔"（composite 注释③的采样点），按启发式归类（活跃层/空闲层）→ 结合 `FrameRateOverrideMappings`（配置的覆盖表）算出全局刷新率提案 → 经 DisplayModeController 切换面板模式。读码三要点：**投票单位是 Layer 不是进程**（一个视频层可把整机拉到 120Hz）；**切换有迟滞**（防抖动，进入高刷要连续 N 个快速帧）；**触摸事件即时提速**（InputFlinger 与 Scheduler 的联动点）——滑动跟手性的最后一环在这里。

## 7.2 RenderEngine：SF 进程内的 GPU 画师

CLIENT 层合成走 `SF/renderengine/skia/`（13 主实现）：Skia 录制各层 draw（纹理采样/混合/圆角裁剪/阴影）→ GPU 扌 行 → 输出帧缓冲层。读码两个要点：**缓存策略**（每层 ExternalTexture 复用 + GLCache 上下文池——合成是每帧全量重画，缓存的是资源不是结果）；**色彩管理**（dataspace 转换在 Skia 管线内完成——HDR/广色域内容在 CLIENT 路径才被正确 tone-map，这是 Overlay 与 CLIENT 画质差异的来源之一）。

# 第八章 Fence：跨设备时序的艺术




图 6 Fence 三主角（acquire/release/present）、内核本质与一帧的 fence 时间轴

Fence（libs/ui/Fence.cpp:48 wait/:90 merge）是内核同步对象的 fd 封装，跨进程经 Binder fd 通道（书3 §六）传递。核心是**不等待的依赖表达**：CPU 提交后即返回，GPU/控制器按依赖图在内核里排队唤醒。三主角：acquire（写完承诺）、release（读完承诺）、present（上屏事实）。排障口诀：**花屏查 release 依赖，掉帧查 fence 延迟轨，"呈现了没"查 present fence**。

# 第九章 设计决策九问

- **为什么缓冲轮转用状态机而非消息？** 槽位是有限资源，状态机天然表达"谁拥有谁负责"，且 FREE/DEQUEUED 归属即锁的边界（第三章）。
- **为什么三缓冲？** 状态机四个角色（写/排/读/垫）的最小完备配置——少则互等、多则纯延迟（第三章）。
- **为什么 BLAST 要事务化帧？** 消除内容/几何双通道错配；帧成为原子单元后回执、预约（desiredPresentTime）、批处理都有了支点（第四章）。
- **为什么 dequeue 返回 fence 而不是同步等？** GPU 流水线深度高，同步等=CPU 陪 GPU 干耗；依赖交给内核，CPU 继续下一帧准备（第三/八章）。
- **为什么 latch 用旧帧？** 帧节拍稳定性的优先级高于单层内容新鲜度（第六章）。
- **为什么 HWC validate 每帧都问？** 层组合能力硬件有限且动态变化（视频进出/透明度变化），静态分配必错（第七章）。

- **为什么 GPU 合成在 SF 进程？** 合成是与 Layer 世界观紧耦合的领域逻辑，进程内免跨进程缓冲搬运；HWC 才是跨进程/跨设备的边界（第七章）。
- **为什么 AIDL BQ 只先落媒体域？** 生产消费跨大进程高频直连只有媒体最痛；UI 域 BLAST 已够（第二/四章）。
- **为什么帧率策略放 SF（LayerHistory）？** 只有合成器看得到全部层的提交历史——投票权在信息最全处（第一章图注/第九章观察）。

# 附录 A 观测实操

| 目标 | 命令/工具 | 看点 |
| --- | --- | --- |
| Layer 全家福 | dumpsys SurfaceFlinger | 每层合成方式(DEVICE/CLIENT)/几何/缓冲状态 |
| 帧时戳 | dumpsys SurfaceFlinger --latency | desiredPresent/present 时间戳序列 |
| 缓冲细节 | dumpsys SurfaceFlinger --dump-buffer | 各槽 fence/时戳（状态机现场） |
| 掉帧归因 | perfetto FrameTimeline | latched/acquire/presented 三段+责任标签 |
| HWC 计划 | dumpsys SurfaceFlinger --list-strategies / 逐层 type | 直通命中情况 |
| App 侧帧 | Choreographer FrameCallback / FrameMetrics | 书5 §四的对接点 |

# 附录 B 快问快答（18 条）

- BQ 数据面走 Binder 吗？——不走：同进程两端+锁；跨进程靠 dma-buf 映射与 AIDL 试点。
- dequeue 与 acquire 的 fence 方向？——dequeue 给 release（等上家读完）；acquire 给 acquire（等生产写完）。
- 三缓冲多分配在哪？——dequeue 分配 GraphicBuffer 按需增长（maxDequeue 限制）。
- BLAST 与旧 BQ 并存吗？——13 默认 BLAST；SurfaceView/媒体路径个别仍旧模型。
- desiredPresentTime 谁消费？——SF commit 侧按预约时间投放（游戏 pacing）。
- Transaction apply 是同步吗？——异步入队；同步需要 explicit 同步模式参数。
- latch 旧帧算丢帧吗？——该层 stale，但不整帧 jank；FrameTimeline 记该层。
- HWC CLIENT 合成产物去哪？——RenderEngine 画进 FB 层再直通。
- Sideband 与 Overlay 区别？——Sideband 绕 SF 合成直叠硬件（低延迟特权）。
- present fence 与 vsync 关系？——present=面板扫描输出事实；vsync=节拍源，两者不重合。
- GraphicBuffer 跨进程映射几次？——按需各进程一次（mapper）；物理内存一份。

- usage 不匹配的后果？——分配在错误堆/无法直通/性能差。
- AIDL BQ 2.0 解决什么？——媒体跨进程直连免中转（13 试点）。
- FrameTimeline 数据源？——BQ 帧事件历史+BLAST 回执+SF 合成记录三方合成。
- 录屏走哪个时钟？——VirtualDisplay 自队（offscreen 结算），非主屏 vsync。
- 帧率投票怎么算？——LayerHistory 各层提交率+heuristic，合成全局刷新率。
- 多屏合成一帧一次吗？——每 DisplayContent 独立 composite。
- 学完本册去哪？——书9 后台与电源（渲染功耗的另一半答案）。

# 附录 C 自检题

- 默画 BufferSlot 状态机与每步携带的 fence。
- BLAST 事务包含哪三类内容？三段回执分别由谁触发？
- composite 内 latch 与 HWC validate 的先后与理由。
- 一帧从 queueBuffer 到 present 的五步时间轴（含三种 fence 落点）。
- 为什么视频场景 GPU 几乎睡觉？用第七章决策三步说明。

# 附录 D 动手实验室（一天量）

实验一"状态机现场"：静止桌面时 `dumpsys SurfaceFlinger --dump-buffer` 记录槽位数与状态；播放视频再看（槽位增长/SHARED 出现）；滚动列表连抓三次（QUEUED/ACQUIRED 波动）——把第三章状态机逐槽对号。

实验二"直通侦查"：播放本地视频时 `dumpsys SurfaceFlinger | grep -A3 video_layer` 找 Composition type=DEVICE；给视频加个半透明遮罩层再看（混合迫使 CLIENT）——第七章决策三步的现场反转。

实验三"fence 时轴"：perfetto 抓 5s 滚动，打开 fence 与 FrameTimeline 轨：对每帧标出 acquire/release/present 三落点；找出一个 Jank(SF) 帧解释哪段超时。

实验四"BLAST 背压"：写 demo 连续 postCallback 高频提交（超出面板刷新率），观察影子队列丢弃（logcat BBQ 标签）与 dequeue 阻塞——第四章两段精读的实证。

实验五"帧率投票"：设置里切 60/120Hz，`dumpsys SurfaceFlinger --frame-rate-override` 对照 LayerHistory 的分层建议；放一个 24fps 视频观察策略（多数机型仍全局高刷、仅部分 LFD 屏做层帧率）。